OpenEMIS Webhook System — Administrator Manual

> **Branch:** POCOR-9257 **Feature:** Async Webhook Queue with Logging and Retry **Scope:** CakePHP (WebhookQueueBehavior) + Laravel API5 (WebhookQueueTrait) + Queue Processor

This manual covers everything you need to connect OpenEMIS to your own systems using webhooks. Whether you want to send a welcome email when a student enrols, sync staff records to an HR platform, or push attendance data to a parent app — webhooks let OpenEMIS notify your system the moment something changes, without any polling.

> **New to webhooks?** Start with Section 1 for a plain-English explanation, then jump to Section 4 to set up your first webhook in the admin UI. Developers building a receiver application should also read Section 14 for working code examples.

Table of Contents

---

1. What Are Webhooks?

Think of a webhook as a tap on the shoulder. Instead of your external system constantly asking “has anything changed?”, OpenEMIS taps your system on the shoulder and says “something just happened — here are the details.” That tap is an HTTP POST request sent to a URL you control.

Common things you might want to do when OpenEMIS taps you:

- **Send a welcome email** when a student is enrolled at a new school
- **Notify HR** when a staff member is hired or leaves
- **Push attendance data** to a parent notification app in real time
- **Sync institution profiles** to a national data warehouse
- **Trigger a workflow** in a case management system when a case is opened

Any time a record is created, updated, or deleted in OpenEMIS, you can have OpenEMIS call your URL with the event details.

What changed in POCOR-9257?

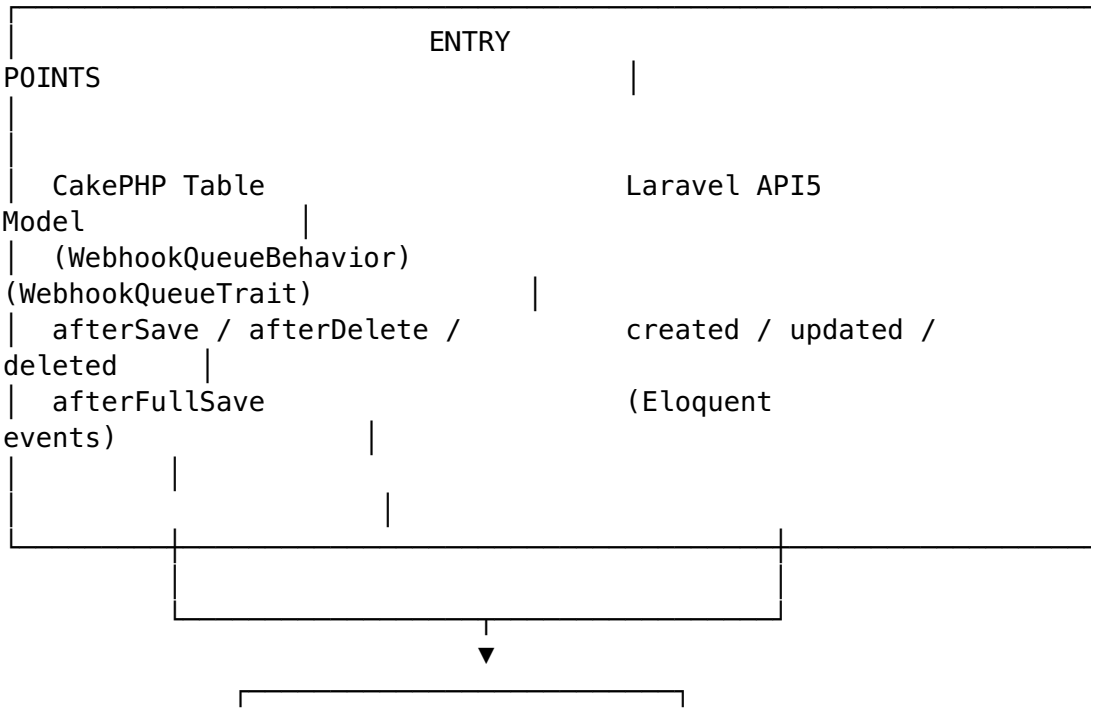
Before: Webhooks fired synchronously — while the user waited. A slow or down endpoint would stall the admin's browser. There was no retry and no record of what was sent.

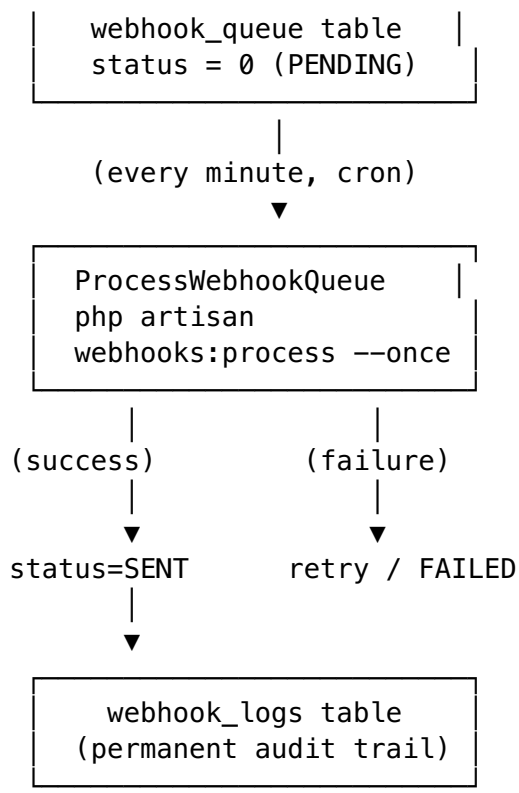
After: Webhooks are queued and delivered in the background, every minute, by a worker process. If delivery fails, it retries automatically with increasing delays. Every attempt — success or failure — is written to a permanent audit log. Webhooks that exhaust all retries can be manually re-sent.

---

2. System Architecture

Under the hood, the webhook system has two separate "entry points" — one for the older CakePHP part of OpenEMIS and one for the newer Laravel API. Both write to the same database queue, and a single background worker picks up that queue and makes the HTTP calls. This means the system works whether a change was made through the admin UI, an API call, or an import.





Key Components

Component	Purpose
`WebhookQueueBehavior`	CakePHP behavior — queues on table save/delete
`WebhookQueueTrait`	Laravel trait — queues on Eloquent events
`WebhookQueueTable`	CakePHP table class for `webhook_queue`
`ProcessWebhookQueue`	Artisan command that delivers queued webhooks
`WebhookSender`	HTTP delivery via Guzzle
`ConfigWebhooksTable`	Admin UI table — webhook rule configuration
`CallWebhookBehavior`	Updated to queue instead of firing directly

File locations:

```
src/Model/Behavior/WebhookQueueBehavior.php
src/Model/Table/WebhookQueueTable.php
api/app/Models/Concerns/WebhookQueueTrait.php
api/app/Console/Commands/ProcessWebhookQueue.php
api/app/Services/WebhookSender.php
plugins/Configuration/src/Model/Table/ConfigWebhooksTable.php
plugins/Configuration/src/Model/Behavior/CallWebhookBehavior.php
```

Dependency: `config_items`

Each webhook rule must be linked to a `config_items` record (the "external data source"). The webhook only fires if both:

1. The webhook rule's `status = 1` (Active)
2. The linked `config_items.value = 1` (Active)

This allows disabling all webhooks for an external system in one action by toggling the config item.

---

3. How It Works — Step by Step

Here is the full journey of a webhook, from a change being made in OpenEMIS to the HTTP call landing at your server.

When someone makes a change in OpenEMIS (CakePHP UI)

1. An admin saves a record — for example, enrolling a student at a school.
2. CakePHP's `WebhookQueueBehavior` intercepts the save and checks: is there an active webhook configured for this event?
3. If yes, it builds the payload (filling in any placeholders you configured) and writes a row to `webhook_queue` with status **PENDING**. That's it — the user's page loads normally.
4. Nothing has been sent to your server yet.

When something changes via the REST API (Laravel)

1. A system calls `POST /api/v5/institutions` to create a new institution.
2. Laravel's Eloquent model fires a `created` event. `WebhookQueueTrait` picks it up.
3. It checks for active webhook rules matching the event key (`institution_create`), builds the payload, and inserts into `webhook_queue` — again, status **PENDING**.
4. The API response returns immediately without waiting for any delivery.

In the background (every minute)

1. The Laravel scheduler triggers `php artisan webhooks:process --once` every minute via cron.
2. It fetches up to 100 pending entries that are ready to send (ordered oldest-first).
3. For each entry — wrapped in a database transaction so a crash in one does not affect the others:
    - Marks it **PROCESSING** (so a parallel run won't pick it up twice).
    - Calls your URL via HTTP using `WebhookSender` (Guzzle under the hood).
    - **If your server responds 2xx or 3xx:** marks it **SENT**, stores the response, writes a log entry.
    - **If delivery fails:** increments the retry count, schedules a retry with exponential backoff (2 min → 4 min → 8 min), writes a log entry. After 3

failures it marks the entry **FAILED** — but you can always re-queue it manually.

---

4. Configuring Webhooks (Admin UI)

Setting up a webhook takes about two minutes. You tell OpenEMIS: *when this type of event happens, call this URL with this payload*. That's it.

Navigate to: **Configuration → External Data → Webhooks** and click **Add**.

Fields at a Glance

Field	Description	Req
Name	Internal label for this webhook	Yes
External Data Source	Linked `config_items` record (must be Active)	Yes
URL	Target endpoint — supports `${placeholder}`	Yes
Event Key	Event that triggers this webhook (§7)	Yes
Method	GET, POST, PUT, PATCH, or DELETE	Yes
Status	Active (1) or Inactive (0)	Yes
Query Template	URL query string with placeholders (§5)	No
Body Template	JSON body with placeholders; empty = full entity	No
Description	Human-readable notes	No

Example Webhook Rule

Field	Value
Name	Student Enrollment Sync
External Data Source	Student Management System (Active)
URL	`https://sms.example.org/api/students`
Event Key	`student_create`
Method	POST
Status	Active
Body Template	`{"student_id": "${openemis_no}", "first_name": "${first_name}"}`

When a student is created in OpenEMIS, this rule fires and POSTs a JSON payload to the configured URL with the student's OpenEMIS ID and name.

---

5. Template System — URL and Body Placeholders

Placeholders let you inject entity field values into the webhook URL and payload at queue time — so the URL your receiver gets is already populated with the right IDs, and the payload contains exactly the fields you care about.

How Placeholders Work

The syntax is `${field_name}`. When OpenEMIS queues a webhook, it scans the URL, query string, and body template and replaces each `${field_name}` with the value of that field from the record that changed.

Placeholder matching works on **flat, top-level field names** from the entity's data. If a placeholder key is not found in the entity, it is left as the literal string `${field_name}` — nothing is silently dropped.

Query Template

The query template is appended to the URL as a query string:

```
URL: https://api.example.org/webhook
Query template: action=${action}&record_id=${id}
Result: https://api.example.org/webhook?
action=student_create&record_id=42
```

Body Template

If the body template is empty, the full entity JSON is sent as the payload.

If a body template is provided, it is treated as a JSON string with placeholder substitution:

```
{
  "event": "student_enrolled",
  "openemis_id": "${openemis_no}",
  "institution_id": "${institution_id}",
  "academic_period_id": "${academic_period_id}"
}
```

The template is processed, then decoded as JSON. If the result is not valid JSON, it is sent as a raw string body.

Delete Events

For delete events, two additional fields are injected into the entity data before placeholder resolution:

Field	Value
`deleted_at`	Timestamp of the delete operation
`deleted_by`	

Field	Value
	`openemis_no`/`username` of the user who deleted, or `"system"`

6. Authentication

Your webhook receiver should be able to verify that requests genuinely came from OpenEMIS. OpenEMIS supports four ways to authenticate outbound webhook requests. The delivery code is fully implemented for all four — UI configuration is planned for a follow-up release.

> **Note:** As of POCOR-9257, auth credentials are stored as `null` in queue entries. Authentication configuration via the admin UI is planned. The delivery code is ready and waiting.

Bearer Token

```
{ "token": "your-bearer-token" }
```

Adds header: `Authorization: Bearer your-bearer-token`

Basic Authentication

```
{ "username": "user", "password": "pass" }
```

Adds header: `Authorization: Basic dXNlcjpwYXNz` (base64 encoded)

API Key

```
{ "key": "your-api-key", "header_name": "X-Custom-Header" }
```

Adds header: `X-Custom-Header: your-api-key` If `header_name` is omitted: `X-API-Key: your-api-key`

HMAC Signature

```
{ "secret": "your-signing-secret" }
```

Adds header: `X-Webhook-Signature: {hmac_sha256(payload, secret)}` Signature is pre-computed at queue time and stored in the `signature` column.

7. Event Keys Reference

Every webhook rule needs an event key — a string that tells OpenEMIS "fire this webhook when *this* type of change happens." When you create a webhook in the admin UI, the Event Key dropdown lists all available keys.

The tables below show every supported key, what triggers it, and whether it comes from the CakePHP or Laravel side of OpenEMIS.

How Event Keys Are Generated

CakePHP path: The `entity_create`, `entity_update`, `entity_delete` values are hardcoded in each table's `addBehavior('WebhookQueue', [...])` call. They match the keys in the admin UI dropdown.

Laravel API5 path: The trait auto-generates the key from the table name: - Default: `{singular_table_name}_{action}` (e.g., `institution_students` → `institution_student_create`) - With `$webhookEventPrefix`: `{prefix}{action}` (e.g., prefix `area_education_` → `area_education_create`)

Full Event Key List

> In the Source column: **CakePHP** = fired via `WebhookQueueBehavior`, **Laravel** = fired via `WebhookQueueTrait` (API5), **Both** = both paths support it.

Institution / School

Event Key	Trigger	Source
`institutions_create`	Institution created	CakePHP
`institutions_update`	Institution updated	CakePHP
`institutions_delete`	Institution deleted	CakePHP

Student

Event Key	Trigger	Source
`student_create`	Student enrolled (institution_students created)	CakePHP
`student_update`	Student record updated	CakePHP
`student_delete`	Student record deleted	CakePHP
`institution_student_create`	Institution student record created	Laravel
`institution_student_update`	Institution student record updated	Laravel

Event Key	Trigger	Source
institution_student_delete	Institution student record deleted	Laravel
attendance_update	Student attendance updated	CakePHP
student_attendance_marked_record_create	Attendance marked record created	Laravel
student_attendance_marked_record_update	Attendance marked record updated	Laravel
student_guardian_create	Student guardian record created	Laravel
student_guardian_update	Student guardian record updated	Laravel
student_guardian_delete	Student guardian record deleted	Laravel

Staff

Event Key	Trigger	Source
staff_create	Staff member created	CakePHP
staff_update	Staff member updated	CakePHP
staff_delete	Staff member deleted	CakePHP
institution_staff_create	Institution staff record created	Laravel
institution_staff_update	Institution staff record updated	Laravel
institution_staff_delete	Institution staff record deleted	Laravel

Class / Subject

Event Key	Trigger	Source
class_create	Class created	CakePHP
class_update	Class updated	CakePHP
class_delete	Class deleted	CakePHP
subject_create	Subject created	CakePHP
subject_update	Subject updated	CakePHP
subject_delete	Subject deleted	CakePHP
institution_class_create	Institution class created	Laravel
institution_class_update	Institution class updated	Laravel
institution_class_delete	Institution class deleted	Laravel

Event Key	Trigger	Source
institution_class_student_create	Student assigned to class	Laravel
institution_class_student_update	Class-student record updated	Laravel
institution_class_student_delete	Student removed from class	Laravel
institution_subject_create	Institution subject created	Laravel
institution_subject_update	Institution subject updated	Laravel
institution_subject_delete	Institution subject deleted	Laravel
institution_grade_create	Institution grade created	Laravel
institution_grade_update	Institution grade updated	Laravel
institution_grade_delete	Institution grade deleted	Laravel

Education Structure

Event Key	Trigger	Source
education_structure_system_update	Education system updated	CakePHP
education_structure_system_delete	Education system deleted	CakePHP
programme_create	Education programme created	CakePHP
programme_update	Education programme updated	CakePHP
programme_delete	Education programme deleted	CakePHP
education_cycle_create	Education cycle created	Both
education_cycle_update	Education cycle updated	Both
education_cycle_delete	Education cycle deleted	Both
education_level_create	Education level created	Both
education_level_update	Education level updated	Both

Event Key	Trigger	Source
`education_level_delete`	Education level deleted	Both
`education_programme_create`	Education programme created	Both
`education_programme_update`	Education programme updated	Both
`education_programme_delete`	Education programme deleted	Both
`education_grade_create`	Education grade created	Both
`education_grade_update`	Education grade updated	Both
`education_grade_delete`	Education grade deleted	Both
`education_subject_create`	Education subject created	Both
`education_subject_update`	Education subject updated	Both
`education_subject_delete`	Education subject deleted	Both
`education_grade_subject_create`	Education grade-subject link created	Both
`education_grade_subject_update`	Education grade-subject link updated	Both
`education_grade_subject_delete`	Education grade-subject link deleted	Both

Academic Period

Event Key	Trigger	Source
`academic_period_create`	Academic period created	Both
`academic_period_update`	Academic period updated	Both
`academic_period_delete`	Academic period deleted	Both

Area / Location

Event Key	Trigger	Source
`area_education_create`	Education area created	Both
`area_education_update`	Education area updated	Both

Event Key	Trigger	Source
`area_education_delete`	Education area deleted	Both

Security / User

Event Key	Trigger	Source
`security_user_delete`	Security user deleted	CakePHP
`security_user_create`	Security user created	Laravel
`security_user_update`	Security user updated	Laravel
`security_user_delete`	Security user deleted	Laravel
`role_create`	Security role created	Both
`role_update`	Security role updated	Both
`role_delete`	Security role deleted	Both
`logout`	User logout	CakePHP

> **Note:** When the same event key appears from both CakePHP and Laravel API5 paths, a single write operation can cause duplicate queue entries if the record is saved through both systems simultaneously. In practice, each path handles different contexts: CakePHP handles UI edits, Laravel API handles API consumers.

---

8. Queue and Retry Behaviour

OpenEMIS doesn't give up on a webhook just because your server was temporarily down. Failed deliveries are automatically retried with increasing delays, giving your endpoint time to recover. Here is how the lifecycle works.

Queue Status Values

Value	Status	Meaning
`0`	PENDING	Waiting to be delivered
`1`	PROCESSING	Currently being sent
`2`	SENT	Delivered successfully
`-1`	FAILED	All retries exhausted

Retry Schedule

When a delivery fails (non-2xx/3xx response, network error, or timeout), the entry is re-queued with an exponential backoff delay:

Retry	Delay	When
1st retry	2 minutes	After first failure
2nd retry	4 minutes	After second failure

Retry	Delay	When
3rd retry	8 minutes	After third failure
Final	status → FAILED	After 3rd retry fails

Default `max_retries = 3`. This is configurable in `api/config/webhooks.php` or via `.env`:

```
WEBHOOK_MAX_RETRIES=3
```

Configuration Parameters

All parameters can be set in `api/.env`:

```
WEBHOOK_TIMEOUT=30                # HTTP request timeout (seconds)
WEBHOOK_CONNECT_TIMEOUT=10        # TCP connection timeout
        (seconds)
WEBHOOK_VERIFY_SSL=true           # Verify SSL certificates
WEBHOOK_MAX_RETRIES=3             # Max delivery attempts
WEBHOOK_BATCH_SIZE=100            # Entries per processing run
WEBHOOK_ENABLED=true              # Master switch
WEBHOOK_LOG_SUCCESS=false         # Log successful deliveries
        (verbose)
WEBHOOK_HMAC_ALGORITHM=sha256     # HMAC signature algorithm
```

Success Criteria

HTTP responses with status `200–399` are treated as success. `4xx` and `5xx` responses are treated as failure and trigger retry logic.

Response Body Truncation

Response bodies are truncated at **10,000 characters** before storage to prevent database overflow. Truncated values end with `... [truncated]`.

---

9. Webhook Logs (Audit Trail)

Every delivery attempt — whether it succeeded or failed — is permanently recorded. This gives you a complete, searchable history of what OpenEMIS sent, when, and what your server responded. The log table is never purged automatically, so you can go back and audit any delivery.

What Is Logged

Field	Description
`webhook_id`	Which webhook rule was triggered
`webhook_queue_id`	The queue entry that was processed

Field	Description
`event_key`	Event that triggered delivery
`target_url`	Exact URL called (after placeholder resolution)
`http_method`	HTTP method used
`payload`	Exact JSON body sent
`headers`	HTTP headers sent
`response_status`	HTTP status code returned
`response_body`	Response body (truncated at 10,000 chars)
`duration_ms`	Round-trip time in milliseconds
`success`	`1` = success, `0` = failure
`error_message`	Error details on failure
`retry_attempt`	`0` = first attempt, `1` = first retry, etc.
`checksum`	SHA256 of `event_key + target_url + payload` for deduplication

Viewing Logs

Navigate to: **Configuration → External Data → Webhook Logs** (read-only view)

Bulk Deleting Log Records

To clean up old log entries via the UI:

1. Navigate to **Configuration → External Data → Webhook Logs**
2. Check the boxes next to the rows you want to remove (or tick the header checkbox to select all visible rows)
3. Click **Delete Selected** — a confirmation dialog will appear
4. Confirm to permanently delete the selected records

> **Note:** `deleteAll` bypasses CakePHP's `beforeDelete`/`afterDelete` callbacks. There are no cascade dependencies on `webhook_logs`, so this is safe.

Or query directly:

```sql
-- Recent deliveries
SELECT id, event_key, target_url, success, response_status,
        duration_ms, created
FROM webhook_logs
ORDER BY created DESC
LIMIT 50;

-- Failed deliveries in the last 24 hours
SELECT id, event_key, target_url, response_status, error_message,
        retry_attempt, created
FROM webhook_logs
WHERE success = 0
  AND created >= NOW() - INTERVAL 24 HOUR
ORDER BY created DESC;
```

```sql
-- All delivery attempts for a specific webhook rule
SELECT l.id, l.success, l.response_status, l.retry_attempt,
       l.duration_ms, l.created
FROM webhook_logs l
WHERE l.webhook_id = <webhook_rule_id>
ORDER BY l.created DESC;
```

---

10. Monitoring and Operations

Cron Setup

The webhook queue processor runs via Laravel's task scheduler. Add one cron entry:

```
* * * * * cd /var/www/html/emis/core/api && php artisan
       schedule:run >> /dev/null 2>&1
```

This runs `webhooks:process --once` every minute. The `--once` flag ensures the command processes one batch and exits cleanly, preventing stacked processes.

Verify the cron is registered:

```
crontab -l
```

Manually Processing the Queue

Navigate to **Configuration → External Data → Webhook Queue** and click the **Process Queue** button to immediately run pending items without waiting for the cron.

Bulk Deleting Queue Records

To purge stuck or unwanted queue entries via the UI:

1. Navigate to **Configuration → External Data → Webhook Queue**
2. Check the boxes next to the rows you want to remove (or tick the header checkbox to select all visible rows)
3. Click **Delete Selected** — a confirmation dialog will appear
4. Confirm to permanently delete the selected records

Use this to clean up permanently-failed entries after investigating them, or to clear the queue during testing.

Checking Queue Depth

```sql
-- Pending (not yet sent)
SELECT COUNT(*) AS pending FROM webhook_queue WHERE status = 0;

-- Failed (all retries exhausted)
SELECT COUNT(*) AS failed FROM webhook_queue WHERE status = -1;
```

```sql
-- By status summary
SELECT
  CASE status
    WHEN 0 THEN 'Pending'
    WHEN 1 THEN 'Processing'
    WHEN 2 THEN 'Sent'
    WHEN -1 THEN 'Failed'
  END AS status_label,
  COUNT(*) AS count
FROM webhook_queue
GROUP BY status;
```

Log Files

```bash
# Laravel logs (all webhook activity)
tail -f /var/www/html/emis/core/api/storage/logs/laravel.log |
        grep -i webhook

# CakePHP error log
tail -f /var/www/html/emis/core/logs/hin-error.log | grep -i
        webhook
```

Key log prefixes: - `[WebhookQueueTrait]` — Laravel model queuing - `[WebhookQueue]` — CakePHP queuing - `[WebhookQueue]` — CakePHP behavior queuing - `[ProcessWebhookQueue]` — delivery processor - `[WebhookSender]` — HTTP request level

Manually Processing the Queue

```bash
# Process one batch (up to 100) and exit
docker exec poe-application /bin/sh -c \
  "cd /var/www/html/emis/core/api && php artisan webhooks:process
        --once"

# Process with custom batch size
docker exec poe-application /bin/sh -c \
  "cd /var/www/html/emis/core/api && php artisan webhooks:process
        --limit=50 --once"
```

Manually Resending Failed Webhooks

Failed webhooks (`status = -1`) can be re-queued by resetting their status:

```sql
-- Resend a specific failed webhook
UPDATE webhook_queue
SET status = 0, retry_count = 0, next_retry_at = NULL, last_error
        = NULL
WHERE id = <queue_id>;

-- Resend all failed webhooks for a specific event
UPDATE webhook_queue
```

```sql
SET status = 0, retry_count = 0, next_retry_at = NULL, last_error
        = NULL
WHERE status = -1 AND event_key = 'institution_update';

-- Resend all failed webhooks
UPDATE webhook_queue
SET status = 0, retry_count = 0, next_retry_at = NULL, last_error
        = NULL
WHERE status = -1;
```

After resetting, the next scheduler run will pick them up.

---

11. Troubleshooting

No Webhooks Firing at All

1. Check that `WEBHOOK_ENABLED=true` is set (or not overridden to `false` in `.env`).
2. Verify the external data source (config item) is **Active** in the admin UI.
3. Verify the webhook rule's **Status** is **Active**.
4. Check that the event key on the rule matches what is being generated. Query the queue:

   ```sql
   SELECT event_key, COUNT(*) FROM webhook_queue
   WHERE created >= NOW() - INTERVAL 1 HOUR
   GROUP BY event_key;
   ```

5. If nothing appears in `webhook_queue`, the entry points are not firing — check CakePHP error log for `[WebhookQueue]` errors.

Webhook Queued but Not Delivered

1. Verify the cron is running: `ps aux | grep "schedule:run"`
2. Manually trigger: `php artisan webhooks:process --once`
3. Check Laravel log for `[ProcessWebhookQueue]` errors.
4. Verify `available_at <= NOW()` on pending entries.

Webhooks Failing with HTTP Errors

1. Query `webhook_logs` for the error details:

   ```sql
   SELECT target_url, response_status, error_message,
           response_body
   FROM webhook_logs
   WHERE success = 0
   ORDER BY created DESC LIMIT 10;
   ```

2. Check that the target URL is reachable from the server:

```
curl -v https://your-endpoint.example.org/webhook
```

3. Check SSL: if the endpoint uses a self-signed certificate, set `WEBHOOK_VERIFY_SSL=false` temporarily.
4. Check timeouts: if the endpoint is slow, increase `WEBHOOK_TIMEOUT` in `.env`.

Placeholders Not Replaced in Payload

Placeholders are only substituted if the field exists as a **top-level key** in the entity data at queue time. Nested relation data is available in the JSON payload but not via placeholder substitution.

Check what fields are available by inspecting a `webhook_queue.payload` value:

```sql
SELECT payload FROM webhook_queue
WHERE event_key = 'institution_update'
ORDER BY created DESC LIMIT 1;
```

Stalled `status = 1` (PROCESSING) Entries

If the processor crashes mid-batch, entries may remain in `status = 1` indefinitely. Safe to reset:

```sql
-- Reset stalled processing entries older than 5 minutes
UPDATE webhook_queue
SET status = 0
WHERE status = 1
  AND modified < NOW() - INTERVAL 5 MINUTE;
```

---

12. Database Schema Reference

`webhook_queue`

Operational queue — holds pending, in-progress, and recently completed webhook deliveries.

Column	Type	Default	Description
`id`	bigint (unsigned)	auto	Primary key
`webhook_id`	int	NULL	References `webhooks.id` (nullable — webhook may be deleted)
`event_key`	varchar(100)	—	Event identifier, e.g. `student_create`
`target_url`	varchar(512)	—	Final URL with placeholders resolved

Column	Type	Default	Description
`http_method`	varchar(10)	POST	GET / POST / PUT / PATCH / DELETE
`headers`	json	NULL	HTTP headers including `Content-Type` and `User-Agent`
`payload`	json	—	Request body
`auth_type`	varchar(20)	NULL	`bearer`, `basic`, `api_key`, `hmac`
`auth_credentials`	json	NULL	Auth credentials
`signature`	varchar(255)	NULL	HMAC signature
`status`	int	0	0=pending, 1=processing, 2=sent, -1=failed
`retry_count`	int	0	Number of attempts made
`max_retries`	int	3	Maximum allowed attempts
`last_error`	text	NULL	Error message from last failed attempt
`available_at`	datetime	CURRENT	Do not process before this time
`next_retry_at`	datetime	NULL	Scheduled retry time (exponential backoff)
`response_status`	int	NULL	Last HTTP response code
`response_body`	text	NULL	Last response body
`duration_ms`	int	NULL	Last request duration in milliseconds
`sent_at`	datetime	NULL	Timestamp of successful delivery
`created`	datetime	CURRENT	When entry was created
`modified`	datetime	NULL	Last status change
`created_user_id`	int	NULL	User who triggered the event

Indexes: `(status, available_at)`, `(event_key)`, `(webhook_id)`, `(next_retry_at)`, `(created)`

`webhook_logs`

Permanent audit trail — every delivery attempt (including retries) is recorded here.

Column	Type	Description
`id`	bigint (unsigned)	Primary key

Column	Type	Description
`webhook_id`	int	References `webhooks.id`
`webhook_queue_id`	bigint	References `webhook_queue.id`
`event_key`	varchar(100)	Event identifier
`target_url`	varchar(512)	URL that was called
`http_method`	varchar(10)	HTTP method used
`payload`	json	Exact body sent
`headers`	json	HTTP headers sent
`response_status`	int	HTTP response code
`response_body`	text	Response body (truncated at 10,000 chars)
`response_headers`	json	Response headers
`duration_ms`	int	Round-trip duration in milliseconds
`success`	boolean	`1` = delivered, `0` = failed
`error_message`	text	Error detail on failure
`retry_attempt`	int	`0` = first attempt, `1` = first retry, etc.
`checksum`	varchar(64)	SHA256 of `event_key + target_url + payload`
`created`	datetime	When this log entry was created
`created_user_id`	int	User who triggered the event

Indexes: `(webhook_id)`, `(webhook_queue_id)`, `(event_key)`, `(checksum)`, `(created)`, `(success)`

---

13. Deployment Instructions

1. Pull the branch

```
git checkout POCOR-9257
git pull origin POCOR-9257
```

2. Run CakePHP Migration

```
cd /var/www/html/emis/core
bin/cake migrations migrate
```

This creates `webhook_queue` and `webhook_logs` tables (backing up existing data if present).

3. Clear Caches

```
# CakePHP
cd /var/www/html/emis/core
```

```
bin/cake cache clear_all

# Laravel
cd /var/www/html/emis/core/api
php artisan config:cache
php artisan route:clear
php artisan cache:clear
```

4. Add Cron Entry

```
crontab -e
# Add:
* * * * * cd /var/www/html/emis/core/api && php artisan
        schedule:run >> /dev/null 2>&1
```

5. Verify

```
# Test queue is empty
docker exec poe-application /bin/sh -c \
  "cd /var/www/html/emis/core/api && php artisan webhooks:process
        --once"

# Test a webhook fires
# Edit any Institution in the UI, then:
mysql -h 127.0.0.1 -P 8136 -u root -prootpassword
        openemis_core_v5 \
  -e "SELECT id, event_key, status, created FROM webhook_queue
        ORDER BY created DESC LIMIT 5;"
```

6. Rollback

If issues occur:

```
cd /var/www/html/emis/core
bin/cake migrations rollback
```

This restores `webhook_queue` and `webhook_logs` to their pre-migration state from the backup tables.

Webhook queueing failures are designed to be **non-blocking** — they are caught and logged, and the parent save operation completes normally. The system degrades gracefully; nothing breaks if webhook delivery is disrupted.

---

14. Integration Examples — Webhook + API

This section shows how an external system can combine OpenEMIS webhooks with the REST API to build real integrations — for example, sending a rich welcome email when a new student is enrolled.

Online API Reference

The full v5 API is documented and browsable online:

https://api.openemis.org/core/v5/

This Swagger UI shows every available endpoint, request parameters, and response schemas. Use it when building integrations to discover exactly which fields are available on each resource.

The Pattern

Webhooks deliver a **trigger** containing the event key and the entity IDs that changed. The webhook payload alone often does not contain every field needed — for example, `institution_student_create` includes `student_id` and `institution_id` but not the student's name or email.

The recommended pattern for rich integrations:

```
1. Receive webhook  →  extract IDs from payload
2. Call OpenEMIS API  →  fetch the full record(s) you need
3. Act  →  send email, sync database, trigger workflow
```

This keeps webhook payloads lightweight while giving integrations access to the complete, up-to-date data.

Authentication — Getting a JWT Token

All v5 API calls require a JWT Bearer token. Obtain one with:

```
POST https://your-openemis-host/api/v4/login
Content-Type: application/json

{
  "username": "api_service_account",
  "password": "your_password"
}
```

Response:

```
{
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "user": { "id": 42, "username": "api_service_account" }
}
```

Include the token in every subsequent API call:

```
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

> **Recommendation:** Create a dedicated read-only service account in OpenEMIS for your integration. Store its credentials in your environment, not in code.

Example 1: New Student Welcome Email (Python)

Scenario: When a student is enrolled at an institution, send a welcome email to the student and CC the institution contact.

Step 1 — Configure the webhook in OpenEMIS Admin

Field	Value
Event Key	`institution_student_create`
URL	`https://your-integration.example.com/webhooks/openemis`
Method	`POST`
Auth	Bearer token issued to your integration app
Body Template	*(leave blank — full payload will be sent)*

The webhook will POST the full `institution_students` record, which includes `student_id` and `institution_id`.

Step 2 — Receive the webhook and call back for details

```python
# webhook_receiver.py
# Requires: flask, requests
# pip install flask requests

import os
import requests
from flask import Flask, request, jsonify
from email.mime.text import MIMEText
import smtplib

app = Flask(__name__)

OPENEMIS_BASE   = "https://your-openemis-host/api/v5"
OPENEMIS_USER   = os.environ["OPENEMIS_API_USER"]
OPENEMIS_PASS   = os.environ["OPENEMIS_API_PASS"]
SMTP_HOST       = os.environ["SMTP_HOST"]
SMTP_PORT       = int(os.environ.get("SMTP_PORT", 587))
SMTP_USER       = os.environ["SMTP_USER"]
SMTP_PASS       = os.environ["SMTP_PASS"]
FROM_ADDRESS    = "noreply@your-ministry.edu"


def get_jwt_token():
    """Authenticate and return a JWT token."""
```

```
    resp = requests.post(
        f"{OPENEMIS_BASE.replace('/v5', '/v4')}/login",
        json={"username": OPENEMIS_USER, "password":
        OPENEMIS_PASS},
        timeout=10,
    )
    resp.raise_for_status()
    return resp.json()["token"]


def api_get(path, token):
    """GET a v5 API resource."""
    resp = requests.get(
        f"{OPENEMIS_BASE}{path}",
        headers={"Authorization": f"Bearer {token}"},
        timeout=10,
    )
    resp.raise_for_status()
    return resp.json().get("data", {})


def send_email(to, subject, body):
    msg = MIMEText(body, "plain")
    msg["Subject"] = subject
    msg["From"]    = FROM_ADDRESS
    msg["To"]      = to
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT) as server:
        server.starttls()
        server.login(SMTP_USER, SMTP_PASS)
        server.sendmail(FROM_ADDRESS, [to], msg.as_string())


@app.route("/webhooks/openemis", methods=["POST"])
def handle_webhook():
    payload = request.get_json(silent=True) or {}
    event   = payload.get("event_key", "")

    # ── Handle: new student enrolled ──────────────────────
    if event == "institution_student_create":
        student_id     = payload.get("student_id")
        institution_id = payload.get("institution_id")

        if not student_id or not institution_id:
            return jsonify({"status": "ignored", "reason":
        "missing IDs"}), 200

        token = get_jwt_token()

        # Fetch full student profile from OpenEMIS v5 API
        student = api_get(f"/security-users/{student_id}", token)

        # Fetch institution details
```

```
        institution = api_get(f"/institutions/{institution_id}",
        token)

        student_name    = f"{student.get('first_name', '')}
        {student.get('last_name', '')}".strip()
        student_email   = student.get("email")
        institution_name = institution.get("name", "your
        institution")
        inst_contact     = institution.get("email")

        if student_email:
            body = (
                f"Dear {student_name},\n\n"
                f"Welcome to {institution_name}!\n\n"
                f"Your OpenEMIS ID is:
        {student.get('openemis_no', 'N/A')}\n\n"
                f"If you have any questions, please contact
        {institution_name} "
                f"at {inst_contact or 'the school office'}.\n\n"
                f"Regards,\nOpenEMIS Administration"
            )
            send_email(student_email, f"Welcome to
        {institution_name}", body)

        return jsonify({"status": "ok"}), 200

    # Acknowledge all other events without processing
    return jsonify({"status": "ignored"}), 200


if __name__ == "__main__":
    app.run(port=8080)
```

Example email produced

```
To: jane.doe@example.com
Subject: Welcome to Westside Primary School

Dear Jane Doe,

Welcome to Westside Primary School!

Your OpenEMIS ID is: 0001-2025-00042

If you have any questions, please contact Westside Primary School
at principal@westside.edu.

Regards,
OpenEMIS Administration
```

Example 2: Staff Assignment Notification (Node.js)

Scenario: Notify a department manager by email whenever a new staff member is assigned to an institution.

Webhook configuration

Field	Value
Event Key	`institution_staff_create`
URL	`https://your-integration.example.com/webhooks/openemis`
Method	`POST`
Body Template	`{"staff_id": "${staff_id}", "institution_id": "${institution_id}"}`

Using a body template here keeps the payload minimal — only the two IDs needed.

Receiver

```
// server.js
// Requires: express, axios, nodemailer
// npm install express axios nodemailer

const express  = require('express');
const axios    = require('axios');
const nodemailer = require('nodemailer');

const app = express();
app.use(express.json());

const OPENEMIS_BASE = process.env.OPENEMIS_BASE || 'https://your-
        openemis-host';
const API_USER      = process.env.OPENEMIS_API_USER;
const API_PASS      = process.env.OPENEMIS_API_PASS;

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT || 587,
  auth: { user: process.env.SMTP_USER, pass:
        process.env.SMTP_PASS },
});

async function getToken() {
  const res = await axios.post(`${OPENEMIS_BASE}/api/v4/login`, {
    username: API_USER,
    password: API_PASS,
  });
  return res.data.token;
}
```

```
async function apiGet(path, token) {
  const res = await axios.get(`${OPENEMIS_BASE}/api/v5${path}`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  return res.data.data;
}

app.post('/webhooks/openemis', async (req, res) => {
  const { event_key, staff_id, institution_id } = req.body;

  if (event_key !== 'institution_staff_create') {
    return res.json({ status: 'ignored' });
  }

  try {
    const token       = await getToken();
    const staff       = await apiGet(`/security-users/${staff_id}
        `, token);
    const institution = await apiGet(`/institutions/$
        {institution_id}`, token);

    const staffName   = [staff.first_name,
        staff.last_name].filter(Boolean).join(' ');
    const instName    = institution.name;
    const instContact = institution.email || 'the institution';

    await transporter.sendMail({
      from:    'noreply@your-ministry.edu',
      to:      instContact,
      subject: `New staff assignment: ${staffName}`,
      text:    `A new staff member has been assigned to $
        {instName}.\n\n`
             + `Name:        ${staffName}\n`
             + `OpenEMIS ID: ${staff.openemis_no}\n`
             + `Email:       ${staff.email || 'not set'}\n\n`
             + `Please review the assignment in OpenEMIS.`,
    });

    res.json({ status: 'ok' });
  } catch (err) {
    console.error('Webhook processing error:', err.message);
    // Return 200 so OpenEMIS does not retry — log the error
        internally
    res.json({ status: 'error', message: err.message });
  }
});

app.listen(8080, () => console.log('Webhook receiver listening on
        :8080'));
```

API Endpoints Used in These Examples

Endpoint	Returns
`POST /api/v4/login`	JWT token
`GET /api/v5/security-users/{id}`	Full user profile (name, email, openemis_no, DOB, etc.)
`GET /api/v5/institutions/{id}`	Institution details (name, address, email, telephone, etc.)
`GET /api/v5/institution-students?student_id={id}`	All enrolment records for a student
`GET /api/v5/institution-staff?staff_id={id}`	All staff assignments for a person

All endpoints accept `?_fields=field1,field2` to limit the returned fields — useful for reducing response size in high-volume integrations.

Browse the full endpoint list at **https://api.openemis.org/core/v5/**

Security Checklist for Integration Receivers

Before going to production, verify your webhook receiver does the following:

- ☐ **Validates the incoming `Authorization` header** — use the bearer token you set in the OpenEMIS webhook configuration to authenticate inbound requests
- ☐ **Returns HTTP 200 quickly** — do expensive work (API calls, email sending) in a background job, not synchronously in the HTTP handler, to avoid timeouts and unnecessary retries
- ☐ **Is idempotent** — OpenEMIS may retry a webhook if your server does not respond in time; your handler should check whether it already processed an event before acting again
- ☐ **Does not expose credentials** — API credentials must be in environment variables, never hardcoded
- ☐ **Uses HTTPS** — OpenEMIS will deliver webhooks only to HTTPS endpoints in production (set `WEBHOOK_VERIFY_SSL=true` in `.env`)